



PEC1

Presentación

Primera PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajan las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre búsqueda informada, su formalización y estrategias relacionadas.

Descripción de la PEC a realizar

1. Disponemos de un tablero con cuatro monedas con la siguiente disposición:

C	X	C	X	
---	---	---	---	--

El anverso de la moneda esta representado por C i el reverso por X.

Podemos realizar los siguientes movimientos:

- **Desplazamiento** (coste = 1): Una moneda puede ser desplazada a la casilla contigua si esta se encuentra vacía.
- **Giro** (coste = 2): Podemos girar cualquier moneda sin ninguna condición adicional.



- **Salto** (coste = 2): Una moneda puede saltar sobre su vecino si a continuación existe una casilla vacía, es decir, solo es posible saltar por encima de una moneda. Cuando una moneda salta, cae girada. Un ejemplo es pasar del estado:

C	X		X	C
---	---	--	---	---

Al estado:

	X	X	X	C
--	---	---	---	---

Nuestro objetivo es el de obtener la **siguiente situación final**:

	X	C	X	C
--	---	---	---	---

Dada la función heurística $h(n) = v + p_2 + p_3 + p_4 + p_5$ en la que p_i tiene valor 0 si la casilla tiene la asignación correcta, el valor 1 en caso contrario y v es la distancia de la casilla en blanco referente a la última casilla. Por ejemplo, el valor de la función heurística del tablero inicial sería de: $h(0) = 4 + 0 + 0 + 0 + 1$.

Es necesario fijarse que en en la primera casilla (p_1) del ejemplo no miramos si la moneda es correcta o no, la heurística valora la distancia que existe de la casilla vacía (posición 0) a la casilla vacía deseada (posición 4).

Otro ejemplo de evaluación de la función heurística para el siguiente estado:

C	X		X	C
---	---	--	---	---

sería $h(0) = 2 + 0 + 1 + 0 + 0$

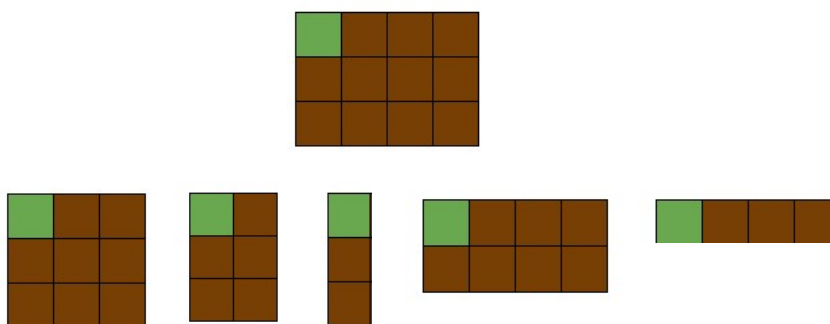


Preguntas

1. Resolver el problema aplicando el algoritmo A* a mano. Indica el orden de expansión de los nodos, el camino obtenido y su coste. (40%)
2. Podemos asegurar que el camino obtenido es de coste óptimo? Es admisible la función h que se propone? Porqué? (40%)
3. Si quisiéramos buscar la solución mediante una búsqueda con coste uniforme, como debería ser el coste de cada jugada para que esta derivase en una búsqueda en amplitud? Podemos asegurar que el camino obtenido es óptimo? (20%)



2. En este problema introduciremos un juego de turnos cuyo tablero es una tableta de chocolate. Nuestro tablero es un rectángulo dividido en porciones pequeñas. Los jugadores siguen un sistema de turnos durante los cuales cada jugador puede coger un conjunto de pastillas que se deben comer. Consideramos esta acción como una jugada
- Las normas del juego indican que el trozo que un jugador debe coger, ha de ser una porción que corte la barra de chocolate **a lo largo o a lo ancho**. La pastilla de chocolate de la esquina superior de la izquierda contiene veneno, el jugador al que le toque comerse esta pastilla, pierde el juego.
- La barra de chocolate de nuestro problema tiene 12 pastillas (3 filas y 4 columnas). A continuación podemos ver el estado inicial de la barra y los 5 posibles movimientos que puede realizar el primer jugador, si nos fijamos, podemos eliminar una o más filas o una o mas columnas.



Un jugador pierde el juego cuando es su turno y la barra de chocolate tiene la siguiente forma:

Es importante hacer notar que este es un juego de tablero tal como lo es el 3 en raya que tenéis en los apuntes de la asignatura. Lo que cambia es que en lugar de llenar un tablero de tamaño fijado, éste se va reduciendo a medida que pasan los turnos y el objetivo del juego es dejar el tablero con una sola pieza, la que hemos dibujado de color verde.

Como podemos esperar, nos interesa estudiar el desarrollo del juego para el jugador 1, aquel que empieza. Consideraremos una función de utilidad en la



que una situación de la tableta vale 1 para el jugador 1 si lo deja en una posición ganadora y -1 en caso contrario.

Preguntas

1. Podemos asegurar que con la función de utilidad propuesta el resultado de la partida sólo depende de las decisiones del primer jugador? Justifica tu respuesta aplicando el algoritmo min-max.(60%)
2. Aplica la poda α - β al árbol que has desarrollado para contestar la pregunta anterior. ¿De qué depende el número de ramas podadas con este procedimiento?(40%)

Soluciones

Problema 1

1. Resolver el problema aplicando el algoritmo A* a mano. Indica le orden de expansión de los nodos, el camino obtenido y su coste.

A partir del estado inicial el cual denotamos como CXCX_ la rama con un coste menor de la función f es la que nos lleva al estado CX_XX mediante la operación salto con el siguiente coste:

$$f(n) = g(n) + h(n) = (2) + (1+0+2+0+1) = 6$$

Por lo tanto es la rama que hay que desarrollar en segundo lugar, esta vez la operación salto será aquella con la que tendremos un coste menor. El resultado del salto es el estado: _XXXX

$$f(n) = (2 + 2) + (0+0+1+0+1) = 6$$

Antes de seguir desarrollando esta rama debemos observar que antes debemos explorar otras ramas que tienen el mismo coste (6).

De todas maneras el camino que nos llevará a la solución final es el que se obtiene de realizar una operación giro _XCXX:



$$fn(n) = (2+2+2) + (0+0+0+0+1) = 7$$

Finalmente un último giro será suficiente para llegar al estado _XCXC

Por lo tanto el coste total será de 8: dos saltos y dos giros.

A continuación tenéis todas las iteraciones del algoritmo con las correspondientes listas de nodos abiertos y cerrados. En esta solución propuesta podéis observar que la selección que se hace de la lista de abiertos en el caso de estados con el mismo coste sigue el criterio de primero en llegar, primero en salir y eso hace que el algoritmo necesite un gran número de iteraciones. También se consideran válidas aquellas soluciones que en caso de empate y debido al conocimiento del problema seleccionen primero los nodos que permiten encontrar una solución con menos iteraciones, concretamente en la iteración 13 vemos que ya tenemos el estado final en la lista de abiertos y que no hay ningún otro estado con un coste menor y por lo tanto podría ser el nodo seleccionado.

Iteración 0

Abiertos(0)=[CXCX_(5)]

Cerrados(0)=[]

Abiertos(1)=[CXC_X(6) CX_XX(6) XXCX_(7) CCCX_(8) CXXX_(8) CXCC_(8)]

Cerrados(1)=[CXCX_]

Abiertos(2)=[CX_XX(6) CXC_C(7) CX_CX(7) CXCC_(8) CCCX_(8) XXCX_(7)
C_CCX(7) XXC_X(8) CCC_X(9) CXX_X(9) CXXX_(8)]

Cerrados(2)=[CXCX_, CXC_X]

Abiertos(3)=[_XXXX(6) C_XXX(7) CXC_C(7) CXCC_(8) CCCX_(8) CX_CX(7)
CX_XC(7) XXC_X(8) CCC_X(9) CXX_X(9) CXXX_(8) XXCX_(7) XX_XX(8)
CC_XX(9) C_CCX(7)]

Cerrados(3)=[CXCX_, CXC_X, CX_XX]

Abiertos(4)=[CXC_C(7) C_XXX(7) CX_CX(7) _XCXX(7) CCCX_(8) C_CCX(7)
CX_XC(7) CXCC_(8) _XXXC(7) CXX_X(9) CXXX_(8) XXCX_(7) XX_XX(8)
CC_XX(9) X_XXX(9) _CXXX(9) XXC_X(8) _XXCX(9) CCC_X(9)]

Cerrados(4)=[CXCX_, CXC_X, CX_XX, _XXXX]

Abiertos(5)=[CX_CX(7) C_XXX(7) CX_XC(7) _XCXX(7) CCCX_(8) C_CCX(7)
CCC_X(9) CXCC_(8) _XXXC(7) C_CCC(8) CXXX_(8) XXCX_(7) XX_XX(8)



```

CC_XX(9) X_XXX(9) _CXXX(9) XXC_X(8) _XXCX(9) CX_CC(8) CXX_X(9)
XXC_C(9) CCC_C(10) CXX_C(10)]
Cerrados(5)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C]
Abiertos(6)=[CX_XC(7) C_XXX(7) C_CCX(7) _XCXX(7) CCCX_(8) XXCX_(7)
CCC_X(9) CXCC_(8) _XXXC(7) C_CCC(8) CXXX_(8) _XXCX(7) XX_XX(8)
CC_XX(9) X_XXX(9) _CXXX(9) XXC_X(8) _XXCX(9) CX_CC(8) CXX_X(9)
XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) CC_CX(10) ]
Cerrados(6)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX]
Abiertos(7)=[C_CCX(7) C_XXX(7) XXCX_(7) _XCXX(7) CCCX_(8) _XXCX(7)
CCC_X(9) CXCC_(8) _XXXC(7) C_CCC(8) CXXX_(8) CC_CX(10) C_XXC(8)
CC_XX(9) X_XXX(9) _CXXX(9) XXC_X(8) _XXCX(9) CX_CC(8) CXX_X(9)
XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XX_XX(8)
XX_XC(9) CC_XC(10) ]
Cerrados(7)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC]
Abiertos(8)=[C_XXX(7) _XCXX(7) XXCX_(7) _XXXC(7) CCCX_(8) _XXCX(7)
_CCCX(7) CXCC_(8) CX_CC(8) C_CCC(8) CXXX_(8) CC_CX(10) C_XXC(8)
CCC_X(9) C_CXX(8) _CXXX(9) XXC_X(8) _XXCX(9) CC_XC(10) CXX_X(9)
XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XX_XX(8)
XX_XC(9) CC_XX(9) X_XXX(9) X_CCX(9) ]
Cerrados(8)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX]
Abiertos(9)=[XXCX_(7) _XCXX(7) _CCCX(7) _XXXC(7) CCCX_(8) _XXCX(7)
_CXXX(7) CXCC_(8) CX_CC(8) C_CCC(8) CXXX_(8) CC_CX(10) C_XXC(8)
CCC_X(9) C_CXX(8) _CXXX(9) XXC_X(8) _XXCX(9) CC_XC(10) CXX_X(9)
XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XX_XX(8)
XX_XC(9) CC_XX(9) X_XXX(9) X_CCX(9) ]
Cerrados(9)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX,
C_XXX]
Abiertos(10)=[_CCCX(7) _XCXX(7) _CXXX(7) _XXXC(7) CCCX_(8) _XXCX(7)
C_CXX(8) CXCC_(8) CX_CC(8) C_CCC(8) CXXX_(8) CC_CX(10) C_XXC(8)
CCC_X(9) X_CCX(9) _CXXX(9) XXC_X(8) _XXCX(9) CC_XC(10) CXX_X(9)
XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XX_XX(8)
XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) XXCC_(10) ]
Cerrados(10)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_]
Abiertos(11)=[_CXXX(7) _XCXX(7) _XXCX(7) _XXXC(7) CCCX_(8) C_XXC(8)
C_CXX(8) CXCC_(8) CX_CC(8) C_CCC(8) CXXX_(8) CC_CX(10) XX_XX(8)
CCC_X(9) X_CCX(9) _XCCX(8) XXC_X(8) _CCCC(8) CC_XC(10) CXX_X(9)
XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10)
XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10)
_CCXX(8) _XXCX(9) ]

```



Cerrados(11)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX]

Abiertos(12)=[_XCXX(7) _XXXC(7) _XXCX(7) CXCC_(8) CCCX_(8) C_XXC(8) C_CXX(8) XXC_X(8) CX_CC(8) C_CCC(8) CXXX_(8) CC_CX(10) XX_XX(8) CCC_X(9) X_CCX(9) _XCCX(8) _CCXX(8) _CCCC(8) CC_XC(10) CXX_X(9) XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10) _XXCX(9) XC_XX(11) _CXXC(8)]

Cerrados(12)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX, _CXXX]

Abiertos(13)=[_XXCX(7) _XXXC(7) C_CXX(8) CXCC_(8) CCCX_(8) C_XXC(8) _CXXC(8) XXC_X(8) CX_CC(8) C_CCC(8) CXXX_(8) CC_CX(10) XX_XX(8) CCC_X(9) X_CCX(9) _XCCX(8) _CCXX(8) _CCCC(8) _XCXC(8) CXX_X(9) XXC_C(9) CCC_C(10) C_XCX(8) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10) _XXCX(9) XC_XX(11) X_CXX(10) CC_XC(10)]

Cerrados(13)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX]

Abiertos(14)=[_XXXC(7) CCCX_(8) C_CXX(8) CXCC_(8) CXXX_(8) C_XXC(8) _CXXC(8) XXC_X(8) CX_CC(8) C_CCC(8) C_XCX(8) CC_CX(10) XX_XX(8) CCC_X(9) X_CCX(9) _XCCX(8) _CCXX(8) _CCCC(8) _XCXC(8) CXX_X(9) XXC_C(9) CCC_C(10) CC_XC(10) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10) _XXCX(9) XC_XX(11) X_CXX(10) X_XCX(10) _XXCC(8)]

Cerrados(14)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX]

Abiertos(15)=[CCCX_(8) CXCC_(8) C_CXX(8) CX_CC(8) CXXX_(8) C_XXC(8) _CXXC(8) XXC_X(8) _CCCC(8) C_CCC(8) C_XCX(8) CC_CX(10) XX_XX(8) CCC_X(9) X_CCX(9) _XCCX(8) _CCXX(8) _XXCC(8) _XCXC(8) CXX_X(9) XXC_C(9) CCC_C(10) CC_XC(10) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10) _XXCX(9) XC_XX(11) X_CXX(10) X_XCX(10) X_XXC(10)]

Cerrados(15)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC]

Abiertos(16)=[CXCC_(8) CXXX_(8) C_CXX(8) CX_CC(8) C_CCC(8) C_XXC(8) _CXXC(8) XXC_X(8) _CCCC(8) CXX_X(9) C_XCX(8) CC_CX(10) XX_XX(8) CCC_X(9) X_CCX(9) _XCCX(8) _CCXX(8) _XXCC(8) _XCXC(8) X_XXC(10) XXC_C(9) CCC_C(10) CC_XC(10) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10) _XXCX(9) XC_XX(11) X_CXX(10) X_XCX(10) CCXX_(11) CCCC_(11)]



```

Cerrados(16)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_]
Abiertos(17)=[CXXX_(8) CX_CC(8) C_CXX(8) XXC_X(8) C_CCC(8) C_XXC(8)
_CXXC(8) _CCXX(8) _CCCC(8) CXX_X(9) C_XCX(8) CC_CX(10) XX_XX(8)
CCC_X(9) X_CCX(9) _XCCX(8) _XXCX(9) _XXCC(8) _XCXC(8) X_XXC(10)
XXC_C(9) CCC_C(10) CC_XC(10) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10)
XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10)
CCCC_(11) XC_XX(11) X_CXX(10) X_XCX(10) CCXX_(11)]
Cerrados(17)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_]
Abiertos(18)=[CX_CC(8) C_CCC(8) C_CXX(8) XXC_X(8) C_XCX(8) C_XXC(8)
_CXXC(8) _CCXX(8) _CCCC(8) CXX_X(9) CCC_C(10) CC_CX(10) XX_XX(8)
CCC_X(9) X_CCX(9) _XCCX(8) _XXCX(9) _XXCC(8) _XCXC(8) X_XXC(10)
XXC_C(9) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10)
XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10)
CCCC_(11) XC_XX(11) X_CXX(10) X_XCX(10)]
Cerrados(18)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_]
Abiertos(19)=[C_CCC(8) C_XCX(8) C_CXX(8) XXC_X(8) CXX_X(9) C_XXC(8)
_CXXC(8) _CCXX(8) _CCCC(8) XXC_C(9) CCC_C(10) CC_CX(10) XX_XX(8)
CCC_X(9) X_CCX(9) _XCCX(8) _XXCX(9) _XXCC(8) _XCXC(8) X_XXC(10)
X_XCX(10) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) XX_CX(9) XXCC_(10)
XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9) _CXCX(10)
CCCC_(11) XC_XX(11) X_CXX(10) C_XCC(9) XX_CC(10) CC_CC(11)]
Cerrados(19)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_, CX_CC]
Abiertos(20)=[C_CXX(8) C_XCX(8) C_XXC(8) XXC_X(8) CXX_X(9) XX_XX(8)
_CXXC(8) _CCXX(8) _CCCC(8) C_CXC(9) CCC_C(10) CC_CX(10) XX_CX(9)
CCC_X(9) X_CCX(9) _XCCX(8) _XXCX(9) _XXCC(8) _XCXC(8) XXC_C(9)
X_XCX(10) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11)
XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9)
_CXCX(10) CCCC_(11) XC_XX(11) X_CXX(10) C_XCC(9) XX_CC(10) X_XXC(10)
X_CCC(10)]
Cerrados(20)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_, CX_CC, C_CCC]

```



Abiertos(21)=[C_XCX(8) XXC_X(8) C_XXC(8) _CCCC(8) CXX_X(9) XX_XX(8)
_CXXC(8) _CCXX(8) _XCXC(8) C_CXC(9) CCC_C(10) CC_CX(10) XX_CX(9)
CCC_X(9) X_CCX(9) _XCCX(8) _XXCX(9) _XXCC(8) C_XCC(9) XXC_C(9)
X_XCX(10) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11)
XXCC_(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9)
CXCX(10) CCCC(11) XC_XX(11) X_CXX(10) X_CCC(10) XX_CC(10)
X_XXC(10)]

Cerrados(21)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_, CX_CC, C_CCC, C_CXX]

Abiertos(22)=[C_XXC(8) XXC_X(8) XX_XX(8) _CCCC(8) _CXCX(8) XX_CX(9)
_CXXC(8) _CCXX(8) _XCXC(8) CXX_X(9) CCC_C(10) CC_CX(10) XXCC_(10)
CCC_X(9) X_CCX(9) _XCCX(8) _XXCX(9) _XXCC(8) C_XCC(9) C_CXC(9)
X_XCX(10) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11)
X_XXC(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) _CXXX(9)
CXCX(10) CCCC(11) XC_XX(11) X_CXX(10) X_CCC(10) XX_CC(10) XXC_C(9)
CCX_X(12)]

Cerrados(22)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_, CX_CC, C_CCC, C_CXX, C_XCX]

Abiertos(23)=[XXC_X(8) _CCCC(8) XX_XX(8) _CCXX(8) _CXCX(8) XX_CX(9)
_CXXC(8) _XCCX(8) _XCXC(8) CXX_X(9) CCC_C(10) CC_CX(10) XXCC_(10)
CCC_X(9) X_CCX(9) _CXXX(9) _XXCX(9) _XXCC(8) C_XCC(9) C_CXC(9)
X_XCX(10) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11)
X_XXC(10) XX_XC(9) CC_XX(9) X_XXX(9) XCCX_(10) XXXX_(10) CCX_X(12)
CXCX(10) CCCC(11) XC_XX(11) X_CXX(10) X_CCC(10) XX_CC(10) XXC_C(9)]

Cerrados(23)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_, CX_CC, C_CCC, C_CXX, C_XCX, C_XXC]

Abiertos(24)=[XX_XX(8) _CCCC(8) _CXXC(8) _CCXX(8) _CXCX(8) XX_CX(9)
CCC_X(9) _XCCX(8) _XCXC(8) CXX_X(9) CCC_C(10) CC_CX(10) XXCC_(10)
CC_XX(9) X_CCX(9) _CXXX(9) _XXCX(9) _XXCC(8) C_XCC(9) C_CXC(9)
X_XCX(10) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11)
X_XXC(10) XX_XC(9) XXC_C(9) X_XXX(9) XCCX_(10) XXXX_(10) CCX_X(12)
CXCX(10) CCCC(11) XC_XX(11) X_CXX(10) X_CCC(10) XX_CC(10)
XCC_X(11) XXX_X(11)]

Cerrados(24)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC,
C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_,
CXCC_, CXXX_, CX_CC, C_CCC, C_CXX, C_XCX, C_XXC, XXC_X]



Abiertos(25)=[_CCCC(8)_CCXX(8)_CXXC(8)_XCCX(8)_CXCX(8)XX_CX(9)
CCC_X(9)_CXXX(9)_XCXC(8)CXX_X(9)CCC_C(10)CC_CX(10)XXCC_(10)
CC_XX(9)X_CCX(9)XXXX_(10)_XXCX(9)_XXCC(8)C_XCC(9)C_CXC(9)
X_XCX(10)CCXX_(11)CC_XC(10)CXX_C(10)CXXC_(11)CC_CC(11)
X_XXC(10)XX_XC(9)XXC_C(9)X_XXX(9)XCCX_(10)XXX_X(11)CCX_X(12)
CXCX(10)CCCC(11)XC_XX(11)X_CXX(10)X_CCC(10)XX_CC(10)
XCC_X(11)]

Cerrados(25)=[CXCX_,CXC_X,CX_XX,_XXXX,CXC_C,CX_CX,CX_XC,
C_CCX,C_XXX,XXCX_,_CCCX,_CXXX,_XCXX,_XXCX,_XXXC,CCCX_,
CXCC_,CXXX_,CX_CC,C_CCC,C_CXX,C_XCX,C_XXC,XXC_X,XX_XX]

Abiertos(26)=[_CCXX(8)_CXCX(8)_CXXC(8)_XCCX(8)CXX_X(9)XX_CX(9)
CCC_X(9)_CXXX(9)_XCXC(8)C_CXC(9)CCC_C(10)CC_CX(10)XXCC_(10)
CC_XX(9)X_CCX(9)XXXX_(10)_XXCX(9)_XXCC(8)C_XCC(9)_XCCC(9)
CCXC(9)CCXX(11)CC_XC(10)CXX_C(10)CXXC_(11)CC_CC(11)X_XXC(10)
XX_XC(9)XXC_C(9)X_XXX(9)XCCX_(10)XXX_X(11)CCX_X(12)_CXCX(10)
CCCC_(11)XC_XX(11)X_CXX(10)X_CCC(10)XX_CC(10)XCC_X(11)
_CXCC(11)X_XCX(10)]

Cerrados(26)=[CXCX_,CXC_X,CX_XX,_XXXX,CXC_C,CX_CX,CX_XC,
C_CCX,C_XXX,XXCX_,_CCCX,_CXXX,_XCXX,_XXCX,_XXXC,CCCX_,
CXCC_,CXXX_,CX_CC,C_CCC,C_CXX,C_XCX,C_XXC,XXC_X,XX_XX,
_CCCC]

Abiertos(27)=[_CXCX(8)_XCCX(8)_CXXC(8)_XCXC(8)CXX_X(9)XX_CX(9)
CCC_X(9)_CXXX(9)_XXCC(8)C_CXC(9)CCC_C(10)CC_CX(10)XXCC_(10)
CC_XX(9)X_CCX(9)XXXX_(10)_XXCX(9)X_CXX(10)C_XCC(9)_XCCC(9)
CCXC(9)CCXX(11)CC_XC(10)CXX_C(10)CXXC_(11)CC_CC(11)X_XXC(10)
XX_XC(9)XXC_C(9)X_XXX(9)XCCX_(10)XXX_X(11)CCX_X(12)_CXCX(10)
CCCC_(11)XC_XX(11)X_XCX(10)X_CCC(10)XX_CC(10)XCC_X(11)
_CXCC(11)]

Cerrados(27)=[CXCX_,CXC_X,CX_XX,_XXXX,CXC_C,CX_CX,CX_XC,
C_CCX,C_XXX,XXCX_,_CCCX,_CXXX,_XCXX,_XXCX,_XXXC,CCCX_,
CXCC_,CXXX_,CX_CC,C_CCC,C_CXX,C_XCX,C_XXC,XXC_X,XX_XX,
_CCCC,_CCXX]

Abiertos(28)=[_CXXC(8)_XCCX(8)CCC_X(9)_XCXC(8)CXX_X(9)XX_CX(9)
CC_XX(9)_CXXX(9)_XXCC(8)C_CXC(9)CCC_C(10)CC_CX(10)XXCC_(10)
XXC_C(9)X_CCX(9)XXXX_(10)_XXCX(9)X_CXX(10)C_XCC(9)_XCCC(9)
CCXC(9)CCXX(11)CC_XC(10)CXX_C(10)CXXC_(11)CC_CC(11)X_XXC(10)
XX_XC(9)_CXCC(11)X_XXX(9)XCCX_(10)XXX_X(11)CCX_X(12)_CXCX(10)
CCCC_(11)XC_XX(11)X_XCX(10)X_CCC(10)XX_CC(10)XCC_X(11)
XC_CX(12)_CXCC(9)]



Cerrados(28)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_, CXCC_, CXXX_, CX_CC, C_CCC, C_CXX, C_XCX, C_XXC, XXC_X, XX_XX, _CCCC, _CCXX, _CXCX]

Abiertos(29)=[_XCCX(8) _XCXC(8) CCC_X(9) _XXCC(8) CXX_X(9) XX_CX(9) CC_XX(9) _CXXX(9) C_XCC(9) C_CXC(9) CCC_C(10) CC_CX(10) XXCC_(10) XXC_C(9) X_CCX(9) XXXX_(10) _XXCX(9) X_CXX(10) _CXCC(9) _XCCC(9) _CCXC(9) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11) X_XXC(10) XX_XC(9) _CXCC(11) X_XXX(9) XCCX_(10) XXX_X(11) CCX_X(12) _CXCX(10) CCCC_(11) XC_XX(11) X_XCX(10) X_CCC(10) XX_CC(10) XCC_X(11) XC_CX(12) XC_XC(12)]

Cerrados(29)=[CXCX_, CXC_X, CX_XX, _XXXX, CXC_C, CX_CX, CX_XC, C_CCX, C_XXX, XXCX_, _CCCX, _CXXX, _XCXX, _XXCX, _XXXC, CCCX_, CXCC_, CXXX_, CX_CC, C_CCC, C_CXX, C_XCX, C_XXC, XXC_X, XX_XX, _CCCC, _CCXX, _CXCX, _CXXC]

Abiertos(30)=[_XCXC(8) _XXCC(8) CCC_X(9) C_XCC(9) CXX_X(9) XX_CX(9) CC_XX(9) _CXXX(9) _CXCC(9) C_CXC(9) CCC_C(10) CC_CX(10) XXCC_(10) XXC_C(9) X_CCX(9) XXXX_(10) _XXCX(9) X_CXX(10) XX_CC(10) _XCCC(9) _CCXC(9) CCXX_(11) CC_XC(10) CXX_C(10) CXXC_(11) CC_CC(11) X_XXC(10) XX_XC(9) _CXCC(11) X_XXX(9) XCCX_(10) XXX_X(11) CCX_X(12) _CXCX(10) CCCC_(11) XC_XX(11) X_XCX(10) X_CCC(10) XC_XC(12) XCC_X(11) XC_CX(12)]

2. Podemos asegurar que el camino obtenido es de coste óptimo? La función h usada es admisible? Porque?

La función usada no es admisible ya que puede sobre-estimar el coste de llegar de un estado al estado objetivo, lo mostraremos con un ejemplo.

De manera similar al ejemplo del rompecabezas no lineal que encontramos en los apuntes de la asignatura cuando se describen algunas heurísticas no admisibles, nos podemos encontrar con la situación de evaluar el siguiente estado X_CXC con valor 2, cuando el coste de llegar a la solución es un desplazamiento con un coste 1.



El problema que presenta nuestra función h es con la estimación de la distancia a la posición vacía. Una posible solución sería usar la parte entera de dividir la distancia de la posición vacía a la posición final entre dos, de esta manera conseguiríamos una estimación optimista.

3. **Si quisiéramos buscar la solución mediante una búsqueda con coste uniforme, como debería ser el coste de cada jugada para que esta derivase en una búsqueda en amplitud? Podemos asegurar que el camino obtenido es óptimo?**

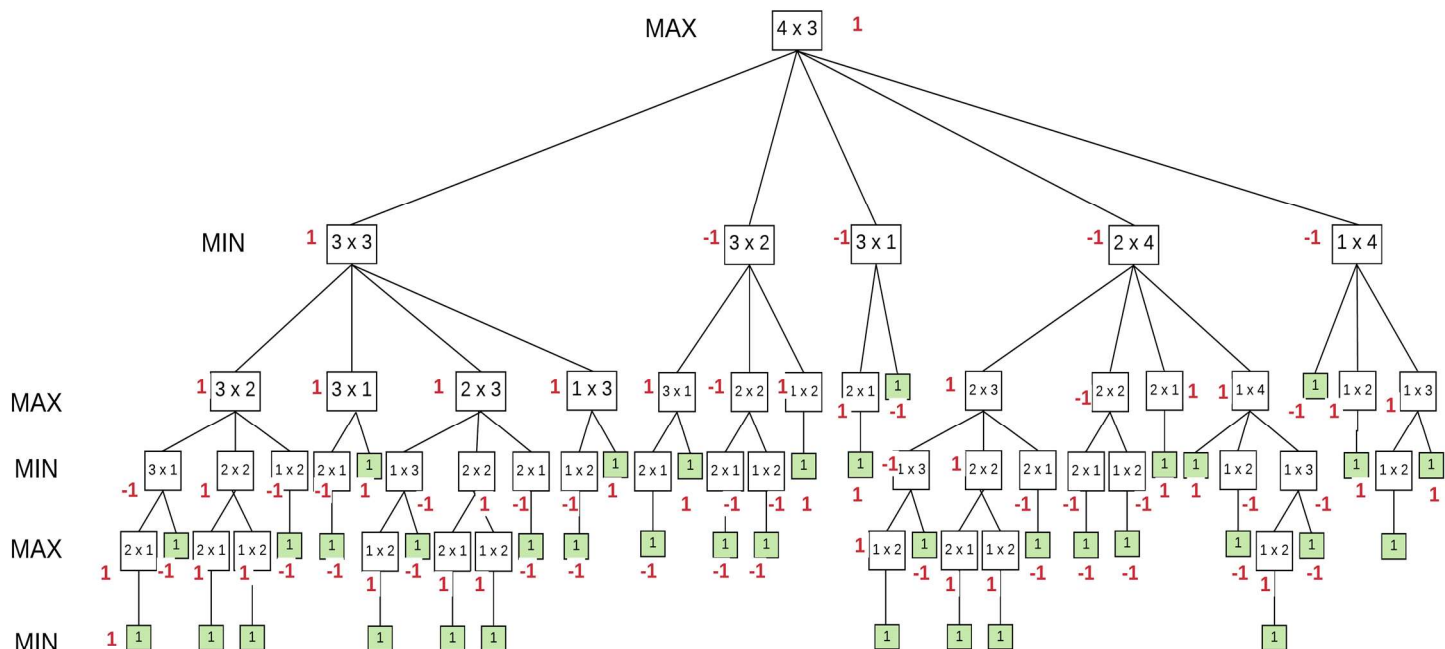
Todas las jugadas (operadores) deberían tener el mismo coste. Si este fuera positivo la búsqueda de coste uniforme es completa y óptima.

Problema 2

1. **Podemos asegurar que con la función de utilidad propuesta el resultado de la partida sólo depende de las decisiones del primer jugador? Justifica tu respuesta aplicando el algoritmo min-max.**

Podemos observar como si el jugador 1 toma las decisiones correctas no hay ningún movimiento que el jugador 2 pueda realizar para no perder la partida.

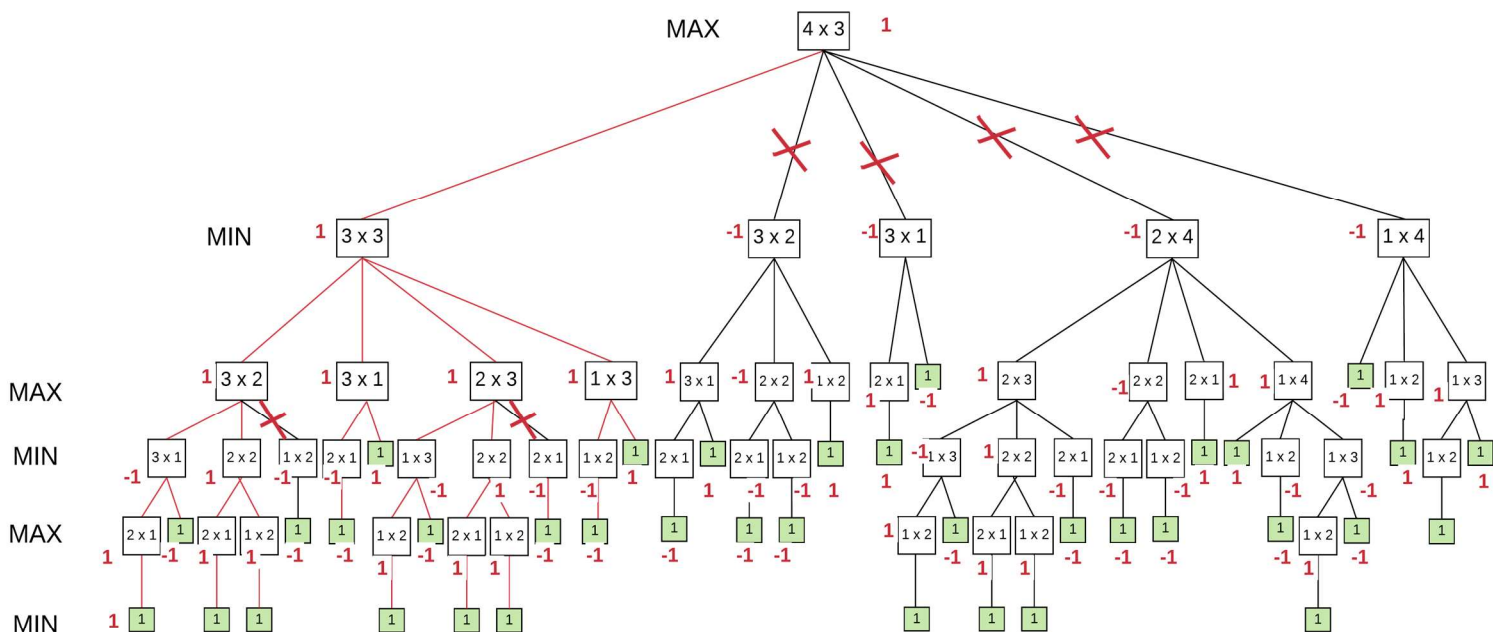
En la siguiente figura tenemos las jugadas que conducen al jugador 1 a ganar la partida. En cada nodo veremos que tenemos dos números, estos se refieren a las filas y las columnas de la tableta resultante. Si nos fijamos, siempre que este deja el tablero con forma cuadrada al segundo jugador, el jugador 1 no puede perder.



2. Aplica la poda α - β al árbol que has desarrollado para contestar la pregunta anterior. ¿De qué depende el número de ramas podadas con este procedimiento?(40%)

En este caso la poda α - β se simplifica por el hecho que solo podemos tener valores binarios y por tanto tal como se muestra en el siguiente gráfico cuando nos encontramos con una situación como la del primer nodo del segundo nivel no podemos encontrar un valor mas alto y podemos podar el resto del árbol. En la figura siguiente, las aristas de color rojo son las que se deben desarrollar.

El número de ramas podadas tiene mucho que ver con el orden de expansión del árbol y por lo tanto si la rama que aparece mas a la izquierda de la figura, se hubiese desarrollado mas tarde, habríamos podido podar menos ramas.



Recursos

Para realizar esta PEC, el material imprescindible es el módulo Resolución de problemas y búsqueda.

También, dentro del paquete de PECs resueltas de semestres anteriores, existen numerosos ejemplos de sistemas difusos.

Criterios de valoración

Las puntuaciones se muestran en cada pregunta del enunciado.



Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, diríjase al consultor responsable de su aula.

Se debe entregar la solución en un archivo PDF utilizando una de las plantillas entregadas conjuntamente con este enunciado. Adjuntar el archivo a un mensaje en el apartado Entrega y Registro de AC (RAC).

El nombre del archivo debe ser *ApellidosNombre_IA_PEC1* con la extensión .pdf (formato PDF).

La fecha límite de entrega es el **22 de Marzo (a las 24 horas)**.

Razona la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...).

El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright.

Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté correspondiente.